

PS159 — Person B Procedure

Dashboard — JSON to Demo, Start to Finish

SecureMailScope

SIH MVP — 2-Day Build

0. Your Job, in One Sentence
1. The Contract You're Building Against
2. Environment Setup (Do This First, ~10 Minutes)
3. Day 1
4. Day 2
5. Fallback Rules — Yours
6. Demo Script — You Drive This
7. Handoff Checklist — What You Need From Person A
8. Judge Questions About Your Half — Know These Cold

0. Your Job, in One Sentence

You take `analysis_result.json` and turn it into something a judge can look at and immediately understand: which sessions are risky, why, and what the evidence is. You never open Person A's Python. They never open your dashboard code. **You should be able to build almost your entire Day 1 without Person A having written anything yet** — that's the whole point of agreeing the schema first and working from mock data.

Mindset while you build this: the numbers and colors you put on screen are what a judge uses to decide whether this tool is real or decorative. A dashboard that shows a fabricated-looking "risk score" with no evidence behind it reads as a mockup. A dashboard where clicking into a HIGH-risk session shows the actual TLS version, the actual expired certificate date, the actual missing STARTTLS upgrade — that reads as a working security tool. Every screen you build should answer "why is this risky?" not just "this is risky."

1. The Contract You're Building Against

Agree this with Person A in the first 30 minutes. You build your entire UI against this shape — you do not need to wait for their real data to start.

```
{
  "metadata": {
    "filename": "weak.pcap",
    "analyzed_at": "2026-09-27T10:00:00Z"
  },
  "summary": {
    "total_sessions": 3,
    "low_risk": 1,
    "medium_risk": 0,
    "high_risk": 2
  },
  "sessions": [
    {
      "session_id": "session_001",
      "protocol": "SMTP",
      "tls": {
        "detected": true,
        "version": "TLS 1.2",
        "cipher_suite": "TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256",
        "key_exchange": "ECDHE"
      },
      "certificate": {
        "present": true,
        "subject": "CN=mail.example.com",
        "issuer": "CN=Test CA",
        "valid_from": "2025-01-01",
        "valid_until": "2026-01-01",
        "expired": false
      },
      "starttls": {
        "attempted": true,
        "succeeded": true,
        "note": null
      },
      "findings": [
        {
          "title": "Modern TLS and valid certificate",
          "severity": "LOW",
          "description": "Session uses TLS 1.2 with a strong cipher suite and a valid certificate.",
          "evidence": "tcp.stream==4",
          "recommendation": "No action needed."
        }
      ],
      "risk": { "level": "LOW", "score": 10 }
    }
  ]
}
```

Rule you must design around: any field can be `null`. Person A will not fabricate data to make your dashboard look complete, so your UI needs to handle `null` gracefully everywhere — show “N/A” or hide the row, never crash, never silently render `None` as text.

2. Environment Setup (Do This First, ~10 Minutes)

```
python3 -m venv venv
source venv/bin/activate
pip install streamlit pandas
```

requirements.txt (your half):

```
streamlit
pandas
```

Why Streamlit and not React+FastAPI: you're one person building a UI in about one working day. Streamlit means you write Python, not a JS build pipeline plus a separate API server plus CORS config. It won't look as polished as a custom React app, but it will work reliably, which is what a feasibility demo needs. (If your team specifically wants React for visual polish and has the hours to spend, this whole section can be redone that way — but that costs real time you don't currently have.)

Repo folders that are yours:

```
PS159-MVP/
├── dashboard/
│   └── app.py
└── sample_results/      # you READ from here – Person A writes here, you never do
```

Git: work on a branch called `dashboard`.

```
git checkout -b dashboard
git commit -m "dashboard: <what you just did>"
```

3. Day 1

Step 1 — Write 3 Mock JSON Files (30 min)

Before Person A has produced anything real, write these by hand, matching the schema exactly:

- `sample_results/mock_good.json` — all LOW risk, modern TLS, valid cert
- `sample_results/mock_weak.json` — mix of LOW/MEDIUM/HIGH, an old TLS version or weak cipher
- `sample_results/mock_critical.json` — a session with `starttls.succeeded: false`, HIGH/CRITICAL risk, maybe an expired cert too

This is your entire test dataset until Person A hands off real output at the end of the day. Build it generously — realistic-looking findings, varied session counts — because your whole UI gets tested against it today.

Step 2 — Dashboard Skeleton

`dashboard/app.py` :

```
import streamlit as st
import pandas as pd
import json
import glob

st.set_page_config(page_title="SecureMailScope", layout="wide")

@st.cache_data
def load_data(path):
    with open(path) as f:
        return json.load(f)

st.sidebar.title("SecureMailScope")

available_files = sorted(glob.glob("sample_results/*.json"))
choice = st.sidebar.selectbox("Load capture", available_files)
data = load_data(choice)

st.title("Email Traffic Security Dashboard")
st.caption(f"File: {data['metadata']['filename']} | Analyzed: {data['metadata']['analyzed_at']}")

col1, col2, col3, col4 = st.columns(4)
col1.metric("Total Sessions", data["summary"]["total_sessions"])
col2.metric("Low Risk", data["summary"]["low_risk"])
col3.metric("Medium Risk", data["summary"]["medium_risk"])
col4.metric("High Risk", data["summary"]["high_risk"])

def safe(val, default="N/A"):
    return val if val is not None else default

rows = []
for s in data["sessions"]:
    rows.append({
        "Session": s["session_id"],
        "Protocol": safe(s.get("protocol")),
        "TLS Version": safe(s["tls"].get("version")),
        "Cert Expired": s["certificate"].get("expired") if s.get("certificate") else "N/A",
        "Risk": s["risk"]["level"],
    })
df = pd.DataFrame(rows)

def highlight_risk(row):
    color = {"LOW": "#d4edda", "MEDIUM": "#fff3cd", "HIGH": "#f8d7da"}.get(row["Risk"], "")
```

```

        return [f"background-color: {color}"] * len(row)

st.dataframe(df.style.apply(highlight_risk, axis=1), use_container_width=True)

session_ids = [s["session_id"] for s in data["sessions"]]
selected_id = st.selectbox("Inspect a session", session_ids)
session = next(s for s in data["sessions"] if s["session_id"] == selected_id)

st.subheader(f"Session Detail – {selected_id}")
c1, c2 = st.columns(2)
with c1:
    st.markdown("**TLS**")
    st.write(session["tls"])
    st.markdown("**STARTTLS**")
    st.write(session.get("starttls", "Not recorded"))
with c2:
    st.markdown("**Certificate**")
    st.write(session["certificate"])

st.markdown("**Findings**")
for finding in session.get("findings", []):
    severity_color = {"LOW": "🟢", "MEDIUM": "🟡", "HIGH": "🔴", "CRITICAL": "⚠️"}.get(finding["severity"], "o")
    with st.expander(f"{severity_color} {finding['title']} – {finding['severity']}"):
        st.write(finding.get("description", ""))
        st.code(finding.get("evidence", "N/A"))
        st.markdown(f"**Recommendation:** {finding.get('recommendation', 'N/A')}")

```

Run with:

```
streamlit run dashboard/app.py
```

Notice this already glob-reads `sample_results/*.json` — so when Person A drops a real file in that folder tonight, it just appears in your dropdown. No code change needed on your end, which is the entire point of the shared-folder contract.

Day 1 — Your Milestone

Dashboard runs against all three mock files, dropdown switches cleanly between them, summary cards, color-coded table, and the expandable findings detail view all work. **At the end of the day, get Person A's real `good.json`, drop it in `sample_results/`, and confirm it renders with zero code changes.** If it doesn't, that's a schema mismatch — flag it to Person A tonight, not tomorrow morning.

4. Day 2

Step 1 — Swap to Real Data

Delete or archive the `mock_*.json` files once Person A's real `good.json` / `weak.json` / `critical.json` exist and render correctly. Your glob-based file list already picks these up automatically.

Step 2 — Live PCAP Upload (with a Safety Net)

```
import subprocess
import tempfile
import json as json_lib

uploaded_file = st.sidebar.file_uploader("Or upload a PCAP directly", type=["pcap", "pcapng"])

if uploaded_file is not None:
    with tempfile.NamedTemporaryFile(suffix=".pcap", delete=False) as tmp:
        tmp.write(uploaded_file.read())
        tmp_path = tmp.name

    if st.sidebar.button("Run live analysis"):
        with st.spinner("Running analyzer..."):
            try:
                result = subprocess.run(
                    ["python", "analyzer/analyzer.py", tmp_path],
                    capture_output=True, text=True, timeout=60
                )
                data = json_lib.loads(result.stdout)
                st.success("Live analysis complete")
            except Exception as e:
                st.error(f"Live analysis failed: {e}. Falling back to precomputed results below.")
```

Critical design point: this is additive, not a replacement. Keep the dropdown-based loading of precomputed JSON files fully working alongside this upload widget. If the subprocess call breaks mid-demo — wrong Python path, tshark not installed on the demo machine, timeout — you switch back to the dropdown without missing a beat. Never make the live path the *only* path.

Step 3 — Polish (Only After Steps 1-2 Are Solid)

- Color-code risk levels in the table (already done in the skeleton above — extend if time allows).
- Make sure every `null` field renders as “N/A” or is hidden, never as literal `None` or a crash.
- Add a one-line summary banner at the top: e.g. “3 sessions analyzed — 2 HIGH risk findings requiring attention.”

Step 4 — Rehearse

Run through the demo script (Section 6) at least twice, with a timer, before you consider Day 2 done.

Step 5 — Git Sync

```
git add dashboard/
git commit -m "dashboard: integrate real analyzer output, add live upload with fallback"
git push origin dashboard
```

Merge to `main` once you've confirmed the dashboard renders Person A's final JSON files with zero further code changes.

5. Fallback Rules — Yours

If this happens	Do this
A field is <code>null</code>	Render “N/A” or hide it — never crash, never show literal <code>None</code>
Live subprocess call to the analyzer fails or times out	Show the error, fall back to the precomputed-JSON dropdown, keep going
Person A’s JSON doesn’t match the schema exactly	Flag it to them immediately — do not quietly patch around it in your dashboard code
You’re behind schedule on Day 2 afternoon	Cut UI polish first. Never cut: the fallback path, the null-safety, or the summary/table/detail views

6. Demo Script — You Drive This

1. Open dashboard, select `good.json` → show LOW risk summary.
2. Select `weak.json` → show HIGH risk summary, visibly different numbers.
3. Click into the high-risk session → walk through TLS version, cipher, certificate status, and the specific finding with its severity and recommendation.
4. Close with: *“This is a 2-day proof of concept for a 2-person subset of the full 6-person SIH proposal — the full system adds AI-assisted risk scoring, STARTTLS-stripping detection, and MTA-STS/DANE policy checks on top of this same pipeline.”*

Success bar: two files in, two visibly different and correct-looking risk outputs out, one real session’s findings inspectable in full detail. Don’t build past that bar on Day 2 — rehearsal time matters more than one more feature.

7. Handoff Checklist — What You Need From Person A

- ☐ `sample_results/good.json` and `weak.json`, real output, matching the schema
- ☐ Confirmation of the exact command to run their analyzer as a subprocess (e.g. `python analyzer/analyzer.py <pcap_path>`)
- ☐ An explicit answer, not a guess, on whether STARTTLS detection is working — so you know whether to render that section or note it as unavailable
- ☐ Their `analyzer` branch pushed, so a fresh clone of `main` actually runs end-to-end

8. Judge Questions About Your Half — Know These Cold

- **“Is this actually running the analysis, or is it hardcoded?”** Show the live upload path first if it’s working, or explicitly say: “the dashboard is decoupled from the analyzer by design — here’s the analyzer running standalone on the command line,” and show that.
- **“What happens if a field is missing?”** It’s rendered as N/A / hidden — nothing on this screen is guessed or filled in to look complete.
- **“Why Streamlit instead of a custom web app?”** Given the 2-day timeframe with one person on the UI, Streamlit let us focus engineering time on making the security analysis correct rather than on frontend plumbing — the full SIH proposal describes a production-grade interface.